Sorting Algorithms

Abdullah All Mamun Siam

Lecturer, Dhaka International University(DIU)

Contents

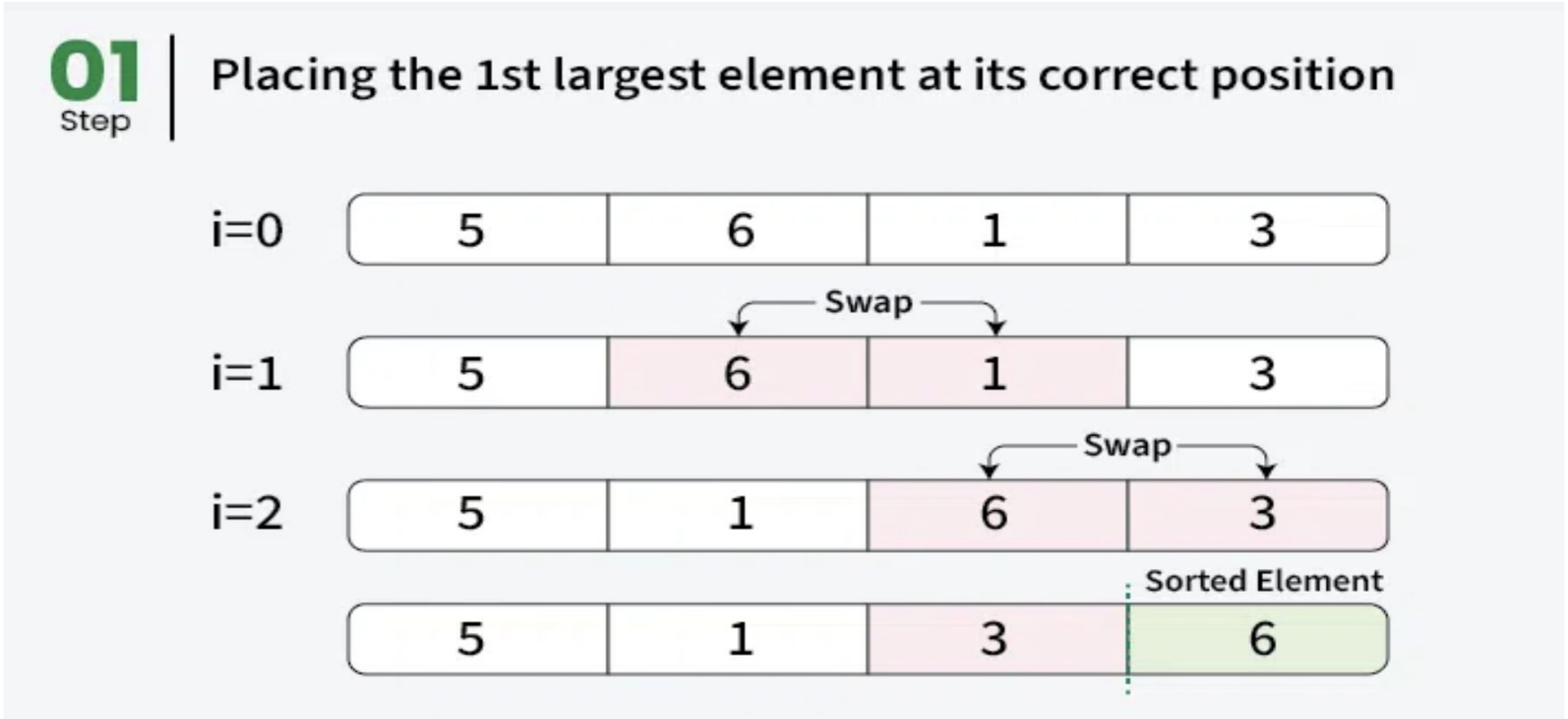
Bubble Sort

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order.

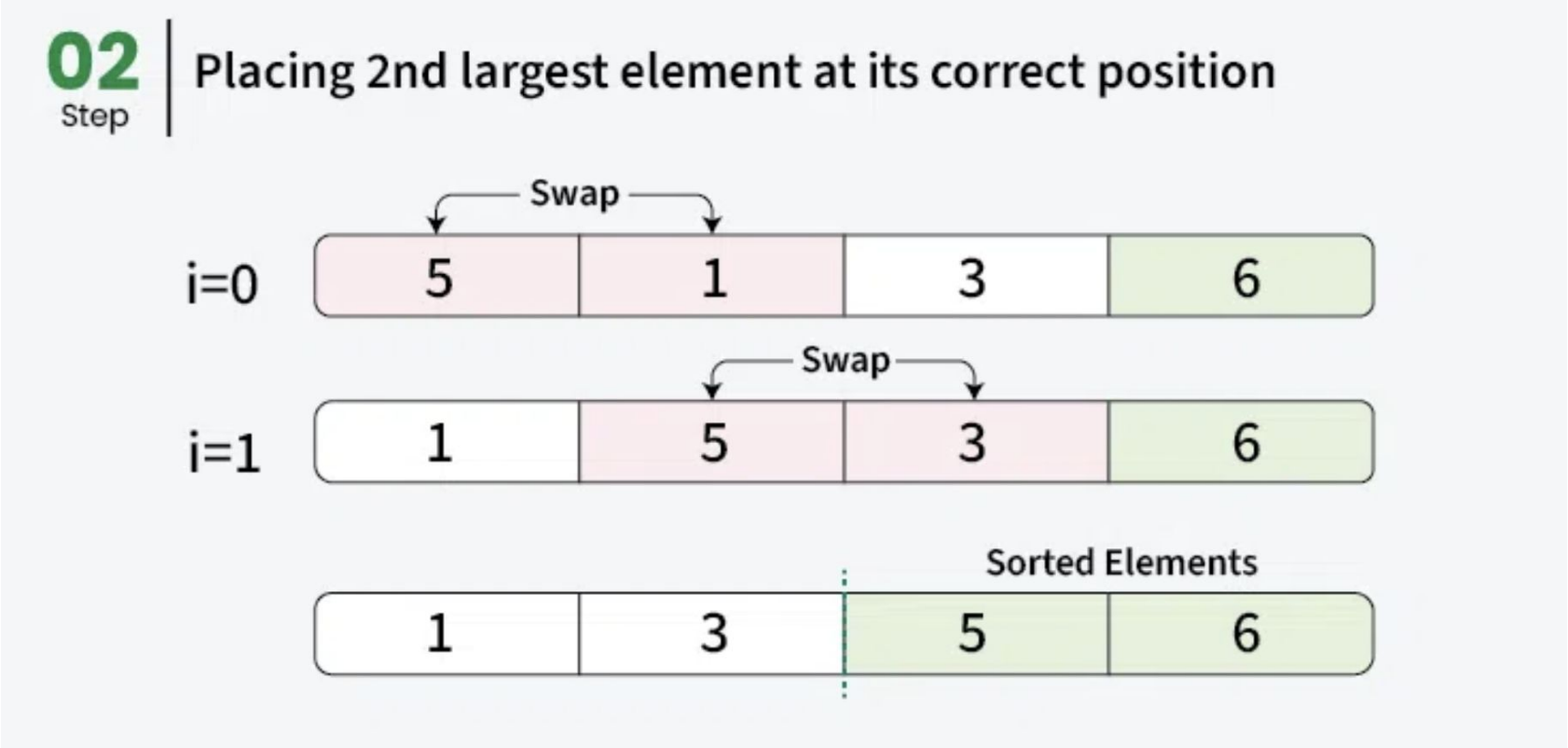
Algorithm Steps:

1. Start from the first element.
2. Compare the current element with the next element.
3. If the current element is greater than the next element, swap them.
4. Move to the next element and repeat step 2 and 3.
5. Repeat the process for the entire list until no swaps are needed.

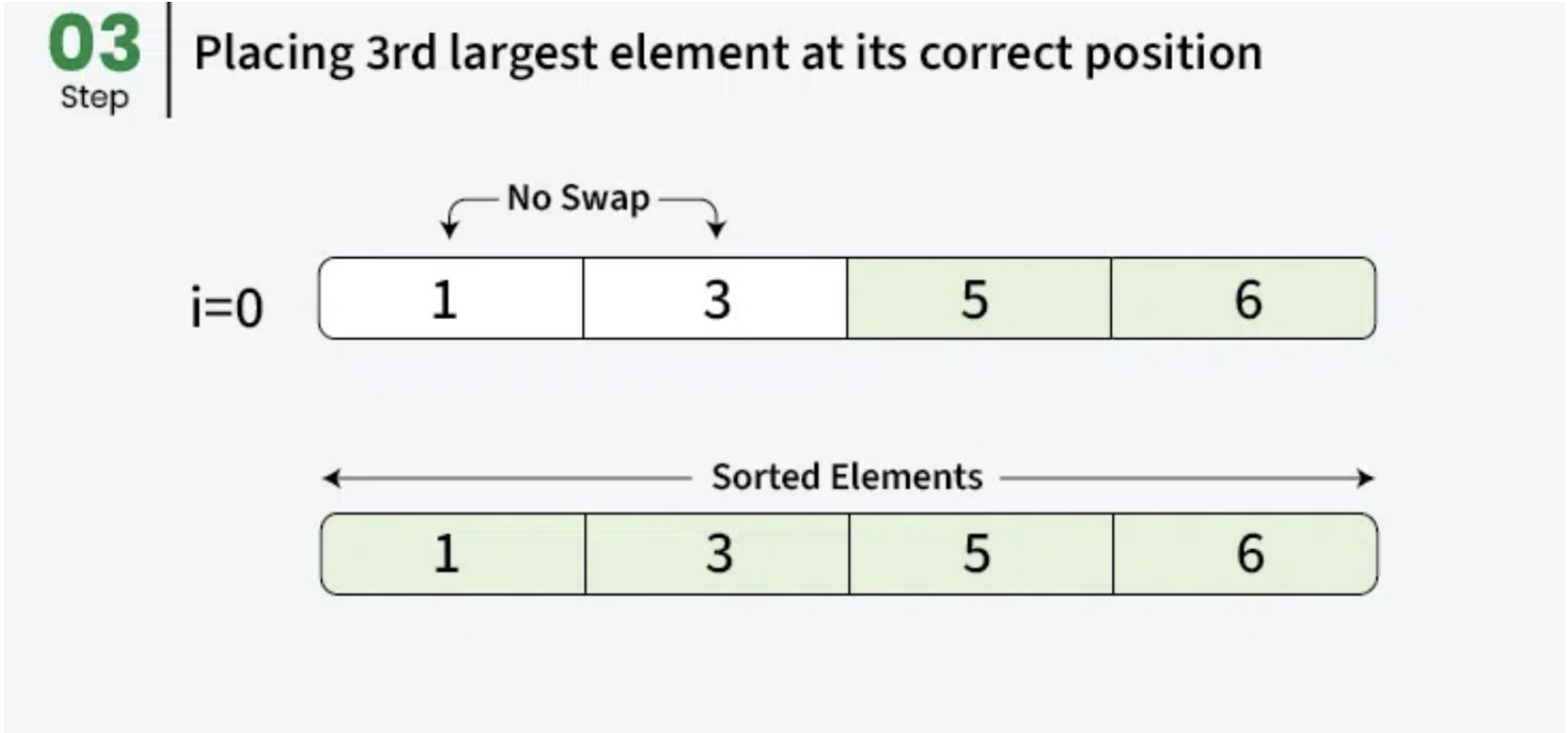
Bubble Sort


01
Step
Placing the 1st largest element at its correct position
i=0
5
6
1
3
Swap
i=1
5
6
1
3
Swap
i=2
5
1
6
3
Sorted Element
5
1
3
6

Bubble Sort


02
Step
Placing 2nd largest element at its correct position
Swap
i=0
5
1
3
6
Swap
i=1
1
5
3
6
Sorted Elements
1
3
5
6

Bubble Sort


03
Step
Placing 3rd largest element at its correct position
No Swap
i=0
1
3
5
6
Sorted Elements
1
3
5
6

Complexity Analysis of Bubble Sort

Time Complexity: $O(n^2)$

Auxiliary Space: O(1)

Advantages of Bubble Sort

1. Bubble sort is easy to understand and implement.
2. It does not require any additional memory space.
3. It is a stable sorting algorithm, meaning that elements with the same key value maintain their relative order in the sorted output.

Disadvantages of Bubble Sort

1. Inefficient for large datasets.
2. High time complexity compared to other sorting algorithms.

Introduction to Merge Sort

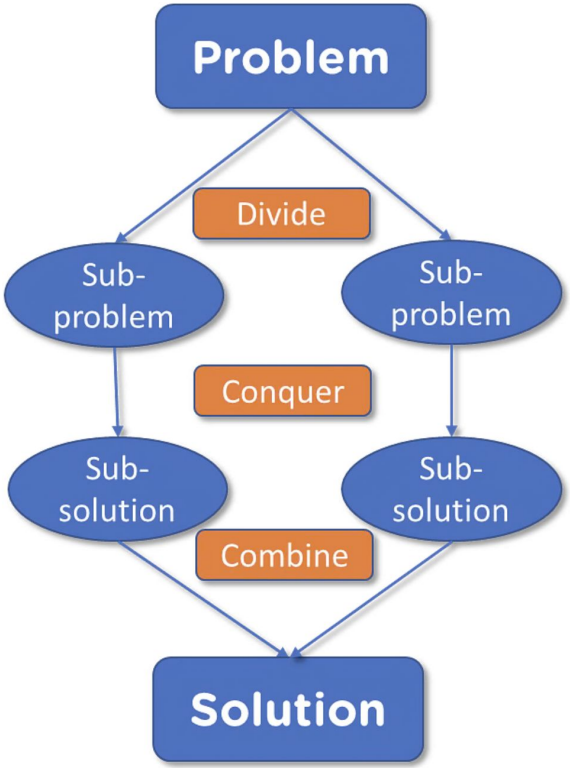
Merge sort is a sorting algorithm that follows the divide-and-conquer approach. It works by recursively dividing the input array into smaller subarrays and sorting those subarrays then merging them back together to obtain the sorted array.

What Is a Divide and Conquer Algorithm?

Divide-and-conquer recursively solves subproblems; each subproblem must be smaller than the original problem, and each must have a base case. A divide-and-conquer algorithm has three parts:

- Divide up the problem into a lot of smaller pieces of the same problem.
- Conquer the subproblems by recursively solving them. Solve the subproblems as base cases if they're small enough.
- To find the solutions to the original problem, combine the solutions of the subproblems.

What Is a Divide and Conquer Algorithm?



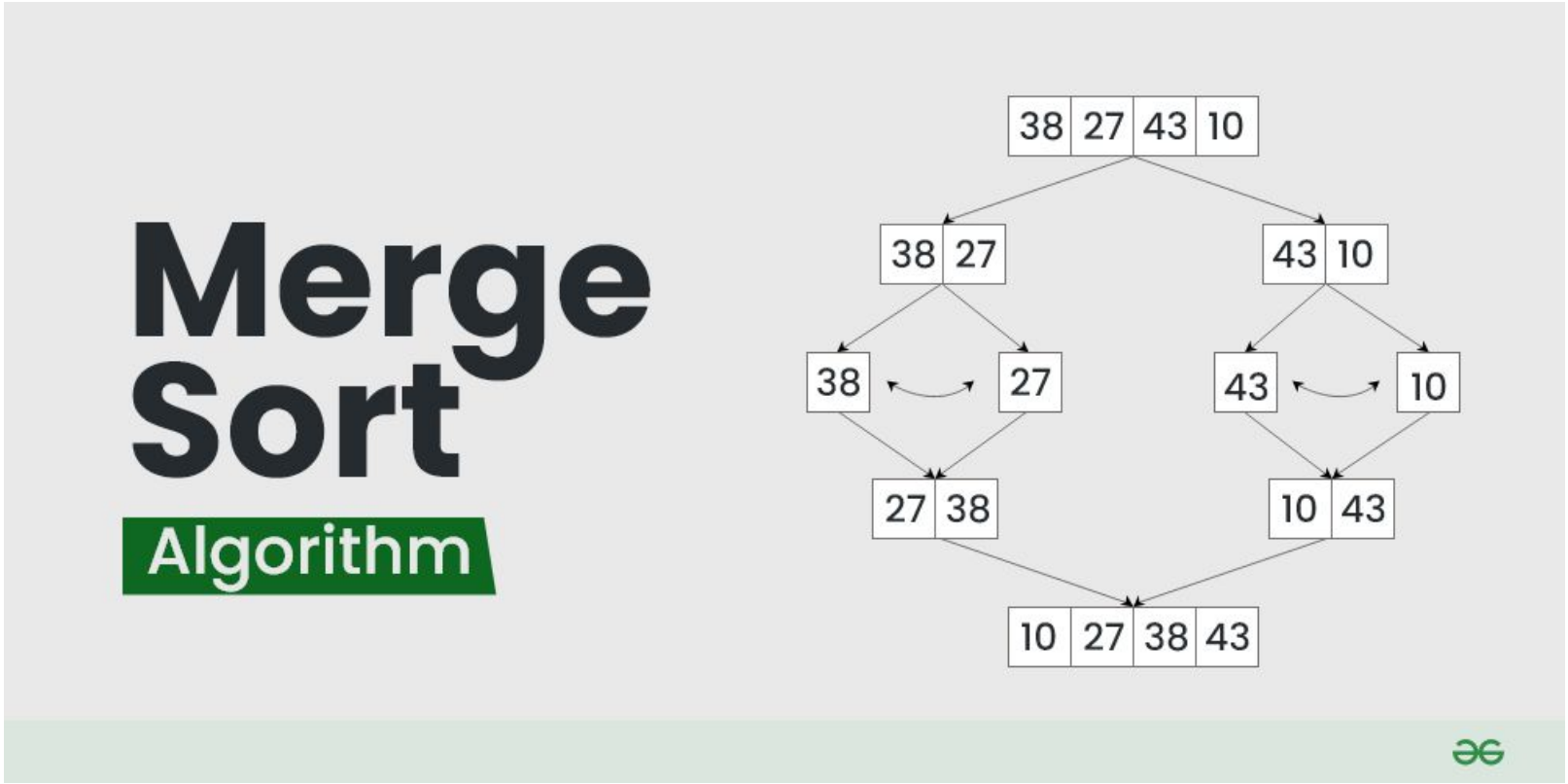
Merge Sort Algorithm Steps

Step 1: **Divide:** The primary step is to repeatedly divide the input list into two halves until you reach sublists containing only one element. A list with one element is considered sorted by definition.

Step 2: **Conquer (Merge):** After dividing, the algorithm starts merging the single-element lists back together in sorted order.

Step 3: **Combine:** The merging process continues until all the sublists are merged into one single sorted list.

Merge Sort Example



Complexity Analysis of Merge Sort

Time Complexity:

Best Case: O(n log n), When the array is already sorted or nearly sorted.

Average Case: O(n log n), When the array is randomly ordered.

Worst Case: O(n log n), When the array is sorted in reverse order.

Auxiliary Space: O(n), Additional space is required for the temporary array used during merging.

Advantages of Merge Sort

1.Merge sort is a stable sorting algorithm, which means it maintains the relative order of equal elements in the input array.

2.Merge sort has a worst-case time complexity of O(N logN) , which means it performs well even on large datasets.

3.The divide-and-conquer approach is straightforward.

Disadvantages of Merge Sort

1. Merge sort requires additional memory to store the merged sub-arrays during the sorting process.
2. Not Merge sort is not an in-place sorting algorithm, which means it requires additional memory to store the sorted data.
3. Merge Sort is Slower than QuickSort

Introduction to Quick Sort

QuickSort is a sorting algorithm based on the Divide and Conquer that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array.

Quick Sort Algorithm Steps

Step1: Choose the pivot (usually first or last element).

Step2: Set left pointer at low, right pointer at high.

Step3: While left ≤ right:

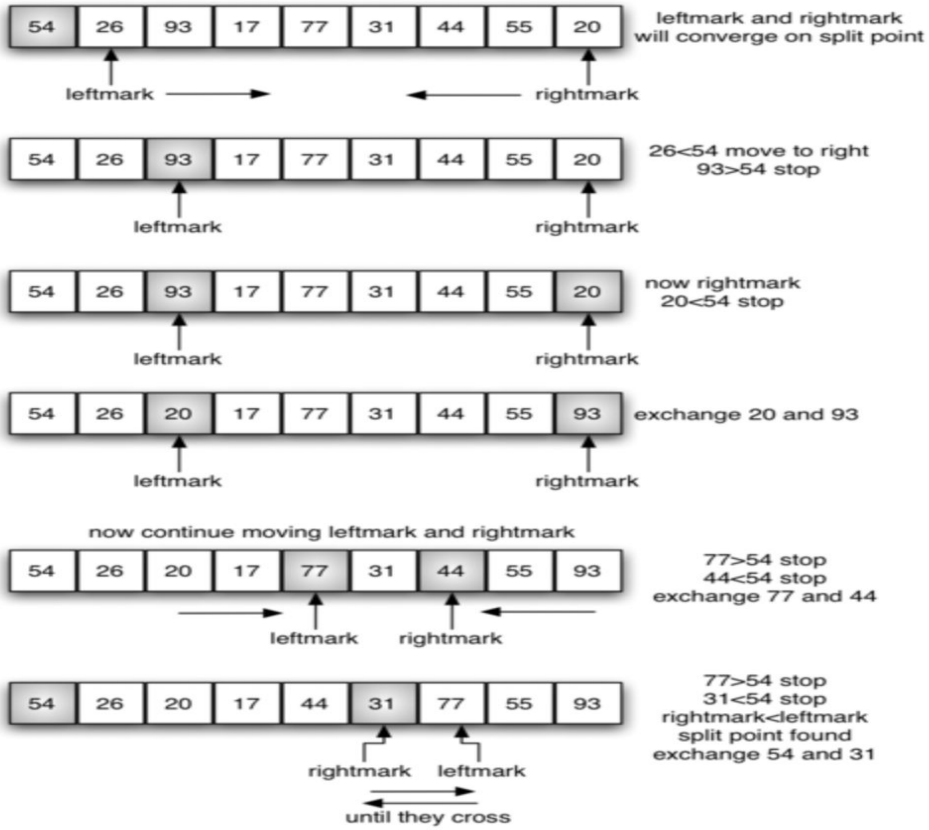
i. Move left right while arr[left] < pivot

ii. Move right left while arr[right] > pivot

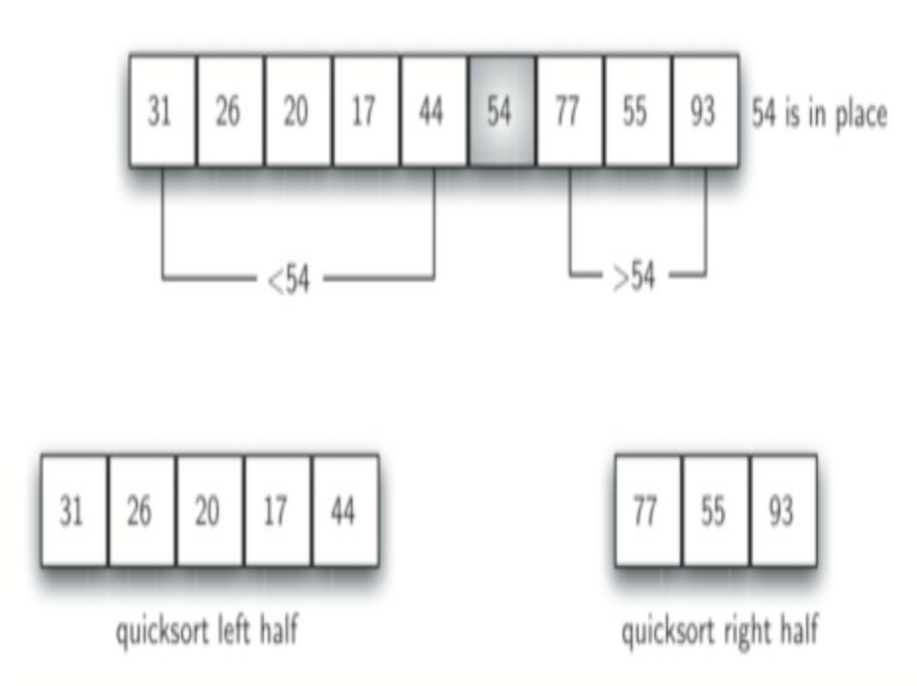
iii. If left < right, swap arr[left] and arr[right].

iv. Otherwise, return the right pointer as the partition index.

Quick Sort Example



Quick Sort Example



Complexity Analysis of Quick Sort

Best Case: O(n log n) (Balanced partitions)

Average Case: O(n log n)

Worst Case: $O(n^2)$ (If pivot is always smallest or largest element)

Space Complexity: O(log n) (Recursive calls stack)

Advantages of Quick Sort

1. It is a divide-and-conquer algorithm that makes it easier to solve problems.
2. It is efficient on large data sets.
3. It has a low overhead, as it only requires a small amount of memory to function.
4. It is Cache Friendly as we work on the same array to sort and do not copy data to any auxiliary array.

Disadvantages of Quick Sort

1. Worst case is $O(n^2)$ (if poor pivot selection).
2. Not stable (relative order of equal elements may change).

Merge Sort Vs Quick Sort

Quick Sort	Merge Sort
1.The splitting of a array of elements is in any ratio, not necessarily divided into half.	1.In the merge sort, the array is parted into just 2 halves
2.Worst case complexity O(n^2)	2.Worst case complexity O(nlogn)
3.Less Additional storage space requirement	3.More Additional storage space requirement
4.Not Stable	4. Stable

Introduction to Insertion Sort

Insertion sort is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list. It is like sorting playing cards in your hands.

You split the cards into two groups: the sorted cards and the unsorted cards. Then, you pick a card from the unsorted group and put it in the right place in the sorted group.

Insertion Sort

Insertion Sort Algorithm Steps:

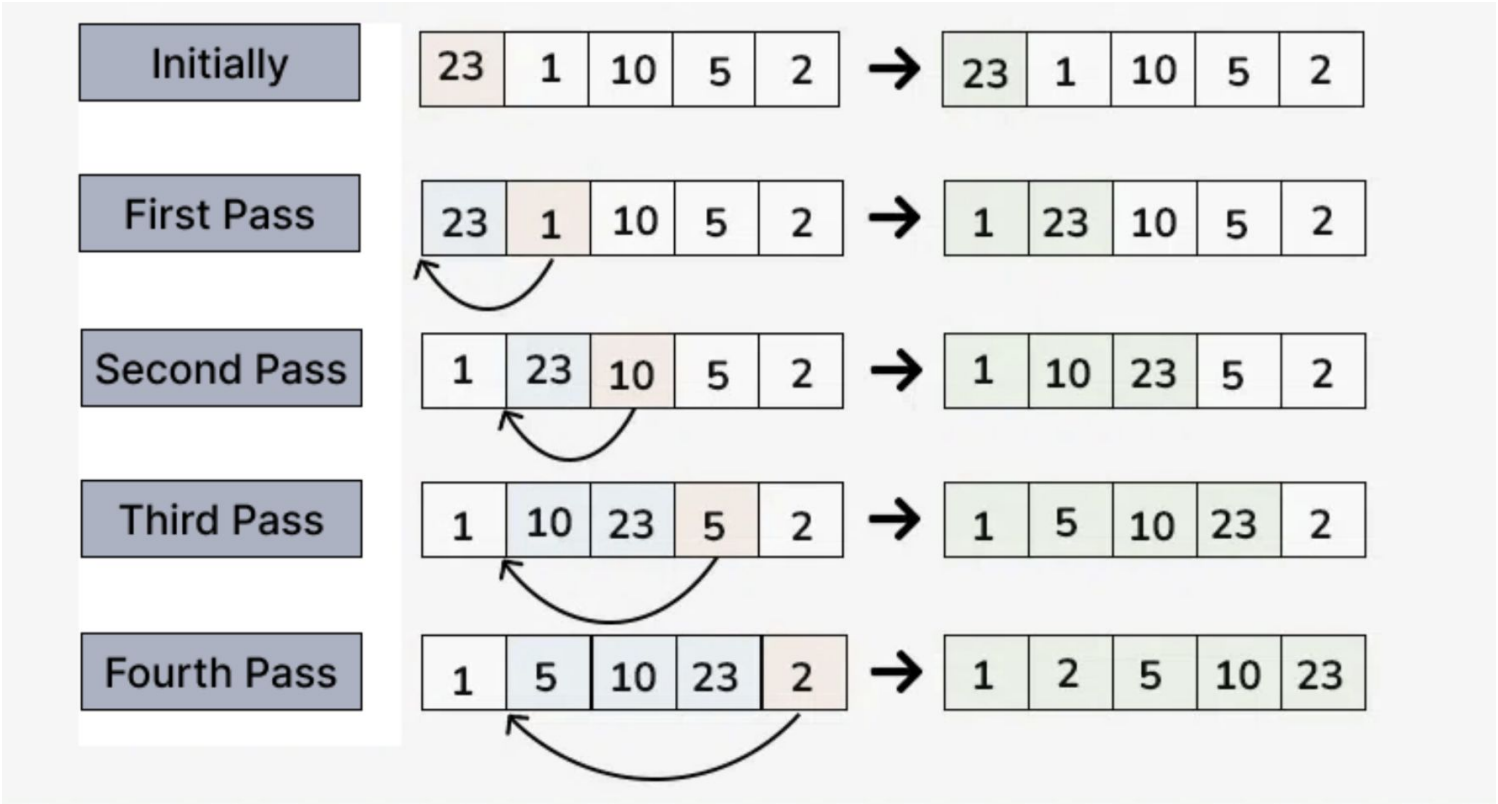
Step 1: We start with second element of the array as first element in the array is assumed to be sorted.

Step 2: Compare second element with the first element and check if the second element is smaller then swap them.

Step 3: Move to the third element and compare it with the first two elements and put at its correct position.

Step 4: Repeat until the entire array is sorted.

Insertion Sort Example



Complexity Analysis of Insertion Sort

Time Complexity of Insertion Sort :

Best case: $O(n)$

Average case: $O(n^2)$

Worst case: $O(n^2)$

Space Complexity of Insertion Sort : Auxiliary Space: $O(1)$

Advantages of Insertion Sort

1.Simple and easy to implement.

2.Stable sorting algorithm.

3.Efficient for small lists and nearly sorted lists.

4.Space-efficient as it is an in-place algorithm.

Disadvantages of Insertion Sort

1.Inefficient for large dataset.

Introduction to Selection Sort

Selection Sort is a comparison-based sorting algorithm. It sorts an array by repeatedly selecting the smallest (or largest) element from the unsorted portion and swapping it with the first unsorted element. This process continues until the entire array is sorted.

Selection Sort

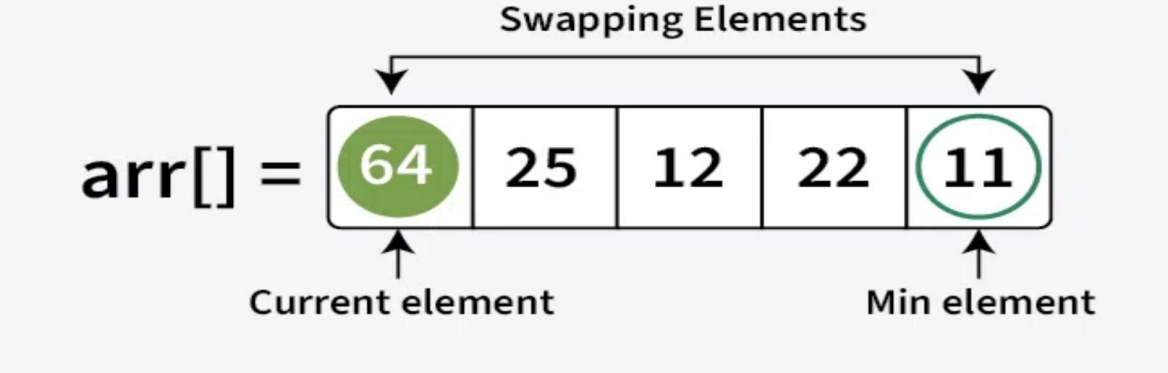
Selection Sort Algorithm Steps:

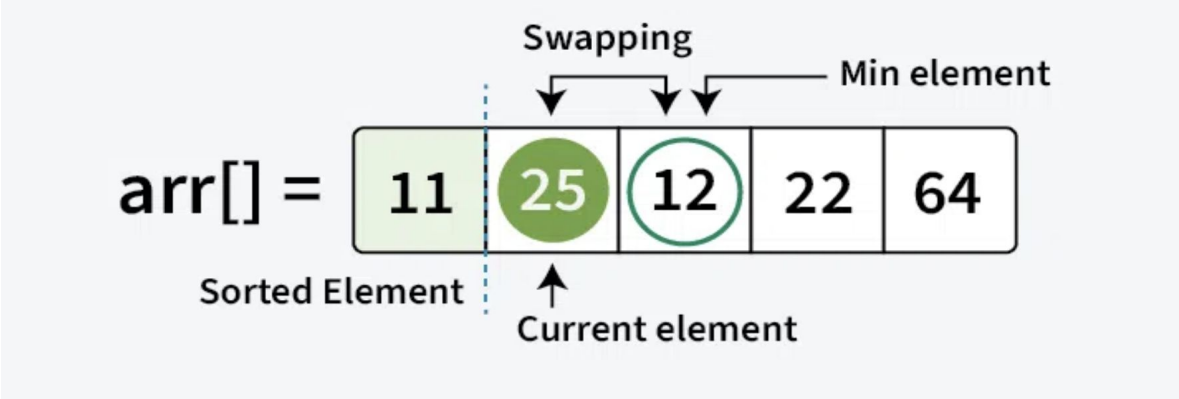
Step 1: First we find the smallest element and swap it with the first element. This way we get the smallest element at its correct position.

Step 2: Then we find the smallest among remaining elements (or second smallest) and swap it with the second element.

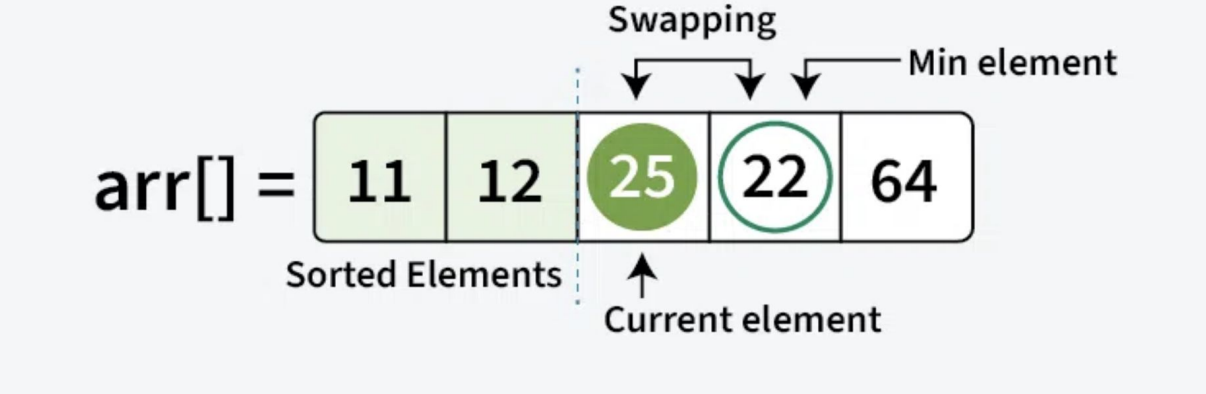
Step 3: We keep doing this until we get all elements moved to correct position.

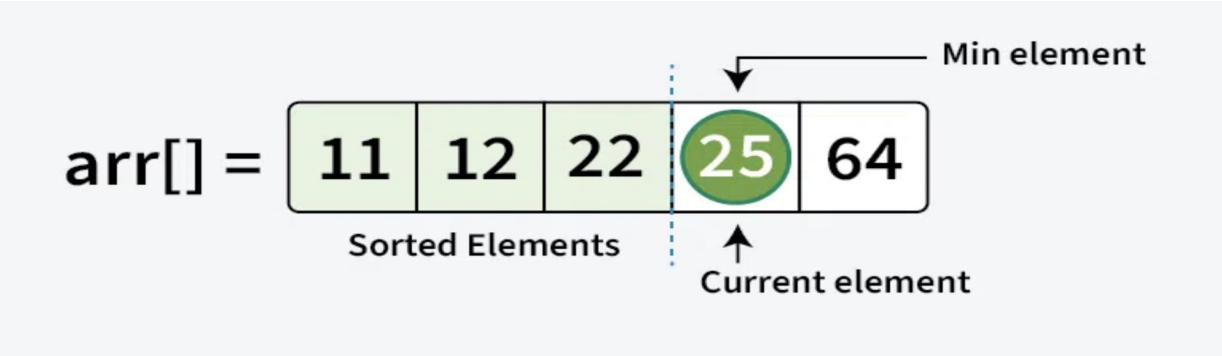
Selection Sort Example



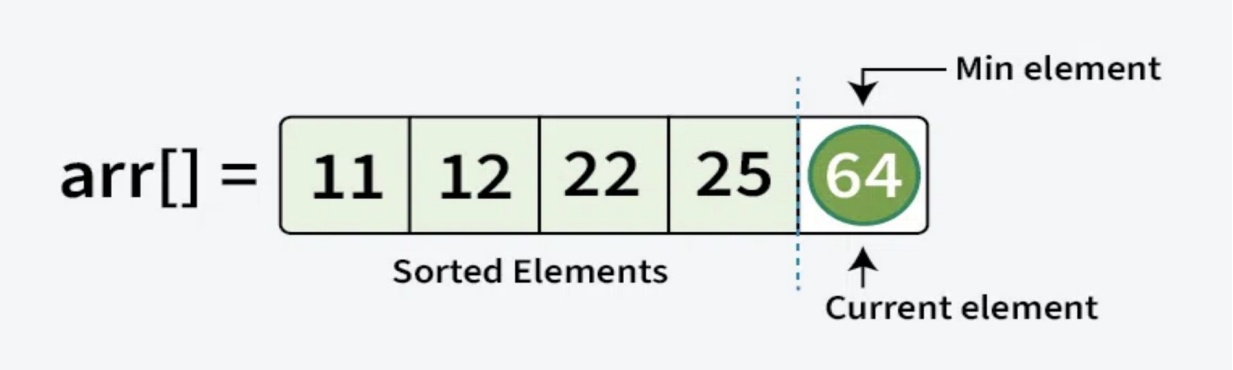


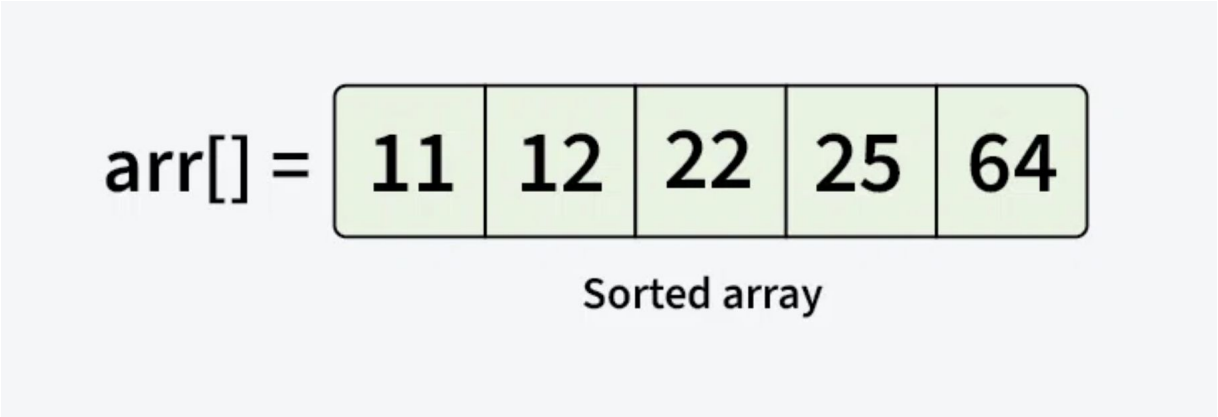
Selection Sort Example





Selection Sort Example





Complexity Analysis of Selection Sort

Time Complexity: $O(n^2)$,as there are two nested loops

Auxiliary Space: O(1)

Advantages of Selection Sort

1.Easy to understand and implement

2.Requires only a constant O(1) extra memory space.

3.It requires less number of swaps (or memory writes) compared to many other standard algorithms.

Disadvantages of the Selection Sort

1.Selection sort has a time complexity of $O(n^2)$ makes it slower compared to algorithms like Quick Sort or Merge Sort.

2.Does not maintain the relative order of equal elements which means it is not stable.

Introduction to Bucket Sort

Bucket Sort is a sorting algorithm that divides elements into multiple **buckets** and then sorts each bucket individually before merging them to produce a sorted array. It is particularly useful for sorting data that is uniformly distributed over a range.

The algorithm typically uses another sorting technique Insertion Sort the individual buckets.

It is commonly used for sorting floating-point numbers and performs efficiently when the input data is evenly spread.

Bucket Sort

Bucket Sort Algorithm Steps:

Step 1 − Divide the interval in n equal parts, each part being referred to as a bucket. Say if n is 10, then there are 10 buckets; otherwise more.

Step 2 − Take the input elements from the input array A and add them to these output buckets B based on the computation formula, B[i]= [n.A[i]]

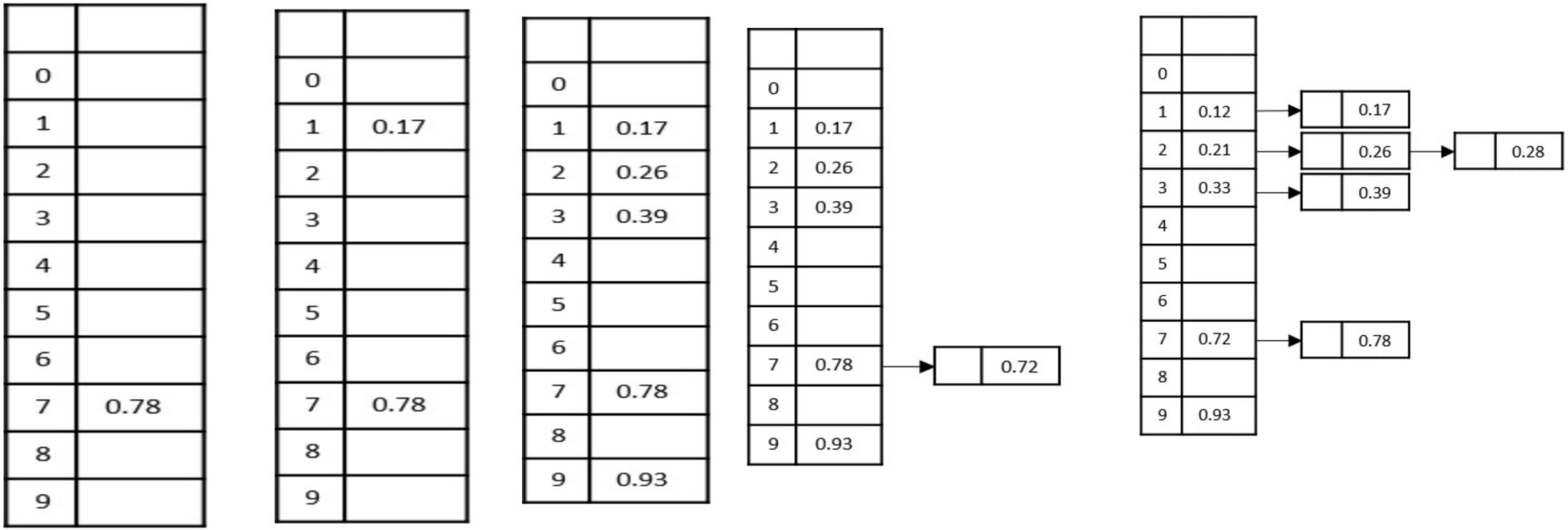
Step 3 − If there are any elements being added to the already occupied buckets, created a linked list through the corresponding bucket.

Step 4 − Then we apply insertion sort to sort all the elements in each bucket.

Step 5 − These buckets are concatenated together which in turn is obtained as the output.

Bucket Sort Example

Consider, an input list of elements: 0.78, 0.17, 0.93, 0.39, 0.26, 0.72, 0.21, 0.12, 0.33, 0.28 to sort these elements using bucket sort –



Concatenate all the buckets together: 0.12, 0.17, 0.21, 0.26, 0.28, 0.33, 0.39, 0.72, 0.78, 0.93

Complexity Analysis of Bucket Sort

Worst Case Time Complexity: $O(n^2)$ The worst case happens when one bucket gets all the elements.

Best Case Time Complexity : O(n + k) The best case happens when every bucket gets equal number of elements.

Auxiliary Space: O(n+k)

Advantages of Bucket Sort

1.Efficient for sorting floating-point numbers and uniformly distributed data.

2.Can achieve linear time complexity in the best case.

3.Works well when the number of buckets is chosen optimally.

Disadvantages of Bucket Sort

1.Performance depends on the distribution of input data.

2.Requires extra space for storing buckets.

3.Choosing the right number of buckets is crucial for efficiency.

Introduction to Radix Sort

Radix Sort is a non-comparative sorting algorithm that sorts numbers by processing individual digits.

It works by distributing each digit, from the least significant to the most significant.

Efficient for sorting large numbers and works well with integer-based data.

Radix Sort

Radix Sort Algorithm Steps:

Step 1 − Find the Maximum Number:

Identify the largest number in the array.

Count the number of digits in this number (this determines the number of sorting passes needed).

Normalize all numbers by treating them as having the same number of digits (by adding leading zeros conceptually).

Step 2 − Sort by Each Digit (LSD to MSD Approach): Start with the least significant digit (rightmost digit). Use a stable sorting method (typically counting sort) to sort numbers based on this digit. Move to the next more significant digit (leftward).

Step 3 − Repeat Until All Digits Are Processed: Continue sorting by digits from right to left until all positions are processed. After the last pass, the array will be fully sorted.

Radix Sort Example

Let the initial array be [121, 432, 564, 23, 1, 45, 788]

Largest value = 788

Number of digits in the largest value = 3

121
432
564
023
001
045
788

Radix Sort Example

Complexity Analysis of Radix Sort

Time Complexity: O(d * (n + b)), where d is the number of digits, n is the number of elements, and b is the base of the number system being used.

Space complexity: O(n + b), where n is the number of elements and b is the base of the number system.

Advantages of Radix Sort

1. Fast for large numbers with a limited number of digits.

2. Works well for integers and fixed-length strings.

Disadvantages of Radix Sort

1.Requires extra space.

2.Performance depends on the number of digits.

Introduction to Counting Sort

Counting Sort is a non-comparison-based sorting algorithm.

It is particularly efficient when the range of input values is small compared to the number of elements to be sorted.

The basic idea behind Counting Sort is to count the frequency of each distinct element in the input array and use that information to place the elements in their correct sorted positions.

Counting Sort

Counting Sort Algorithm Steps:

Step 1 − Maintain two arrays, one with the size of input elements without repetition to store the count values and other with the size of the input array to store the output.

Step 2 − Initialize the count array with all zeroes and keep the output array empty.

Step 3 − Every time an element occurs in the input list, increment the corresponding counter value by 1, until it reaches the end of the input list.

Step 4 − Now, in the output array, every time a counter is greater than 0, add the element at its respective index.

Step 5 − Repeat Step 4 until all the counter values become 0. The list obtained is the output list.

Counting Sort Example

Consider an input list to be sorted, 0, 2, 1, 4, 6, 2, 1, 1, 0, 3, 7, 7, 9.

Counter Array

0	1	2	3	4	6	7	9
0	0	0	0	0	0	0	0

Output

0	1	2	3	4	5	6	7	8	9	10	11	12

Counting Sort Example

Consider an input list to be sorted, 0, 2, 1, 4, 6, 2, 1, 1, 0, 3, 7, 7, 9.

Counter Array

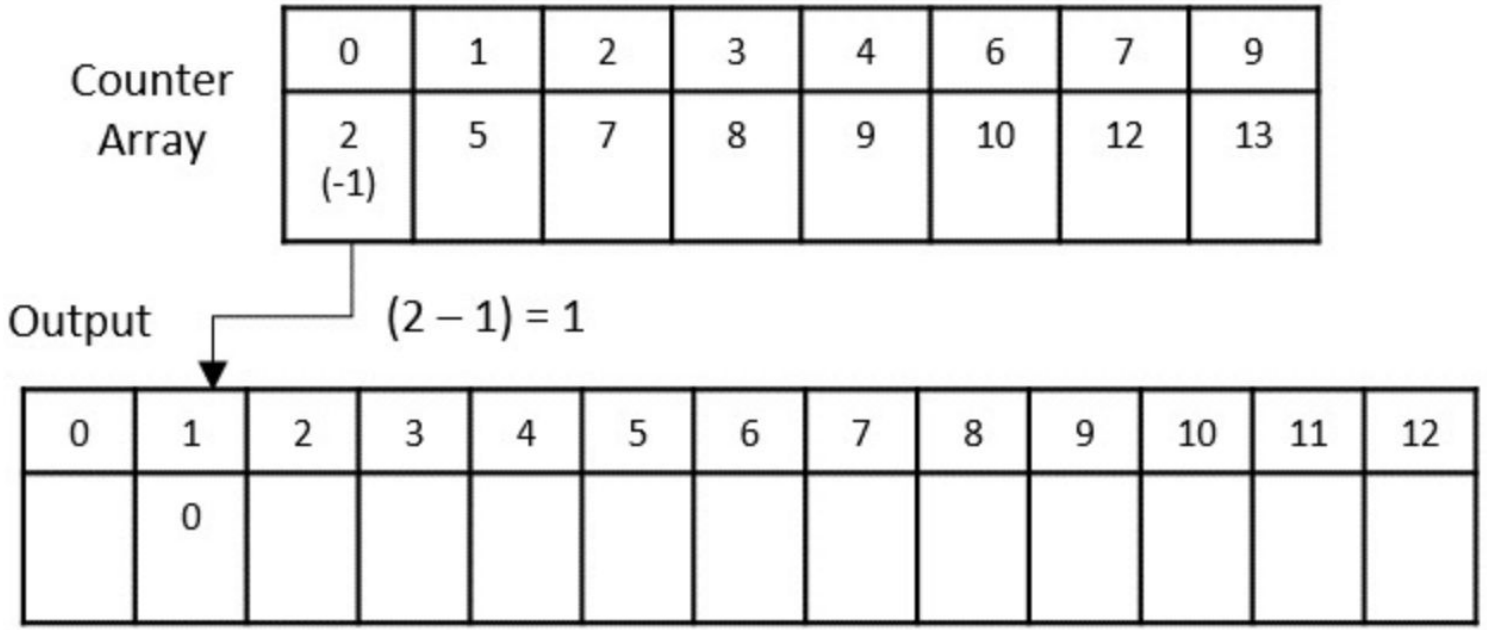
0	1	2	3	4	6	7	9
2	5	7	8	9	10	12	13

Output

0	1	2	3	4	5	6	7	8	9	10	11	12

Counting Sort Example

Consider an input list to be sorted, 0, 2, 1, 4, 6, 2, 1, 1, 0, 3, 7, 7, 9.



Counting Sort Example

Consider an input list to be sorted, 0, 2, 1, 4, 6, 2, 1, 1, 0, 3, 7, 7, 9.

Counter Array

0	1	2	3	4	6	7	9
2 (-1)	5 (-1)	7 (-1)	8 (-1)	9 (-1)	10 (-1)	12 (-1)	13 (-1)

Output

0	1	2	3	4	5	6	7	8	9	10	11	12
	0			1		2	3	4	6		7	9

Counting Sort Example

Consider an input list to be sorted, 0, 2, 1, 4, 6, 2, 1, 1, 0, 3, 7, 7, 9.

Counter Array

0	1	2	3	4	6	7	9
1 (-1)	4 (-1)	6 (-1)	7	8	9	11 (-1)	12

Output

0	1	2	3	4	5	6	7	8	9	10	11	12
0	0		1	1	2	2	3	4	6	7	7	9

Counting Sort Example

Consider an input list to be sorted, 0, 2, 1, 4, 6, 2, 1, 1, 0, 3, 7, 7, 9.

Counter Array

0	1	2	3	4	6	7	9
1	3 (-1)	5	7	8	9	10	12

Output

0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	1	1	1	2	2	3	4	6	7	7	9

The final sorted output is achieved as 0, 0, 1, 1, 1, 2, 2, 3, 4, 6, 7, 7, 9

Complexity Analysis of Counting Sort

Time Complexity: O(N+M), where N and M are the size of inputArray[] and countArray[] respectively.

Worst-case: O(N+M).

Average-case: O(N+M).

Best-case: O(N+M).

Auxiliary Space: O(N+M), where N and M are the space taken by outputArray[] and countArray[] respectively.

Advantages of Counting Sort

1.Extremely fast for small-range integer sorting.

2.Stable sorting algorithm (preserves the relative order of equal elements).

3.Works well when dealing with categorical or integer data.

Disadvantages of Counting Sort

1.Inefficient for sorting large numbers with a wide range.

2.Requires extra space.

3.Not suitable for sorting floating-point number..

Introduction to Heap Sort

Heap Sort is an efficient sorting technique based on the heap data structure.

The heap is a nearly-complete binary tree where the parent node could either be minimum or maximum.

The heap with minimum root node is called min-heap and the root node with maximum root node is called max-heap.

The elements in the input data of the heap sort algorithm are processed using these two methods.

Heap Sort

Heap Sort Algorithm Steps:

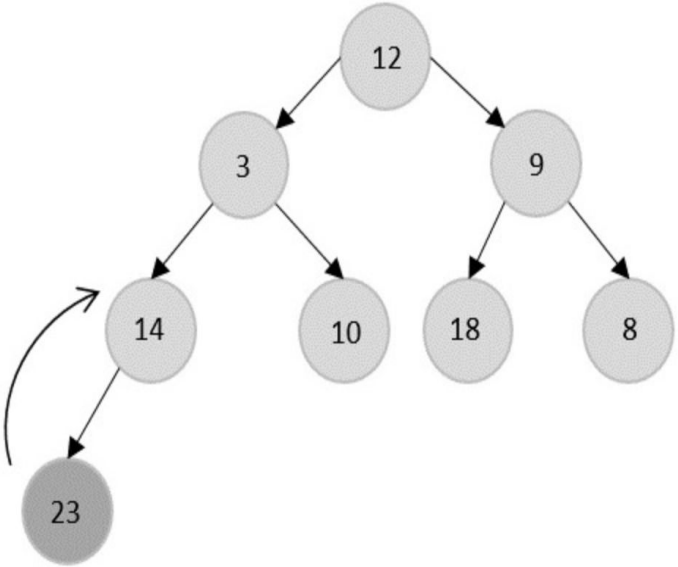
Step 1 − Build a Max Heap.

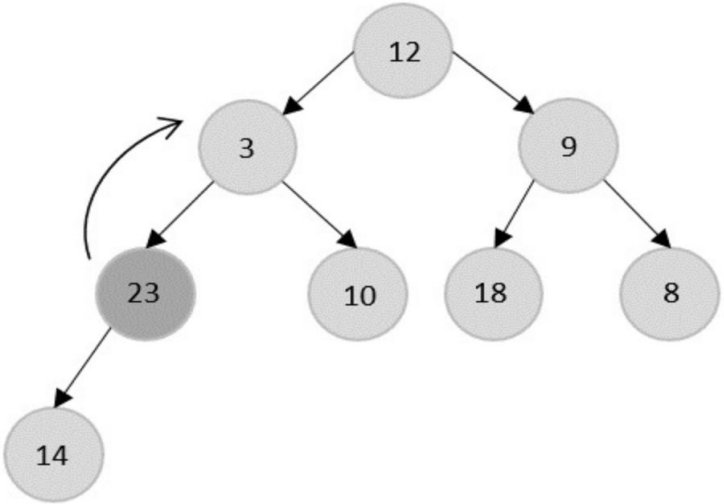
Step 2 − Extract Elements from the Heap.

Step 3 − Repeat Step 2 until the heap size becomes 0.

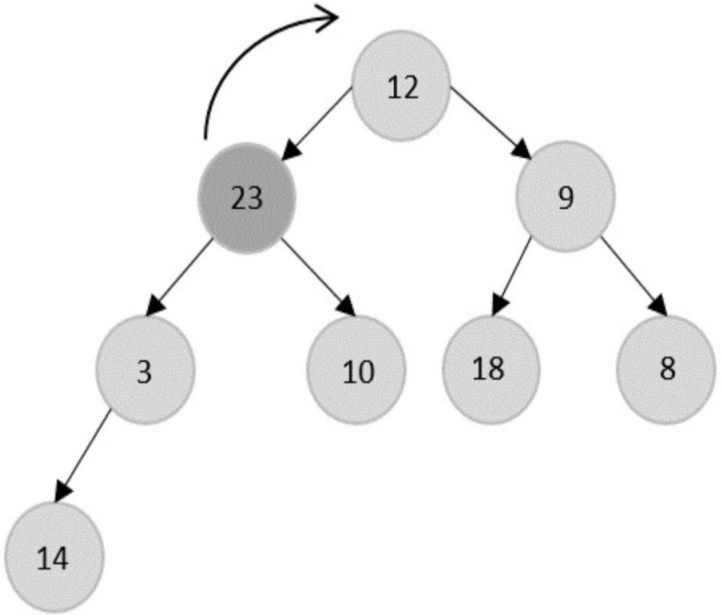
Heap Sort Example

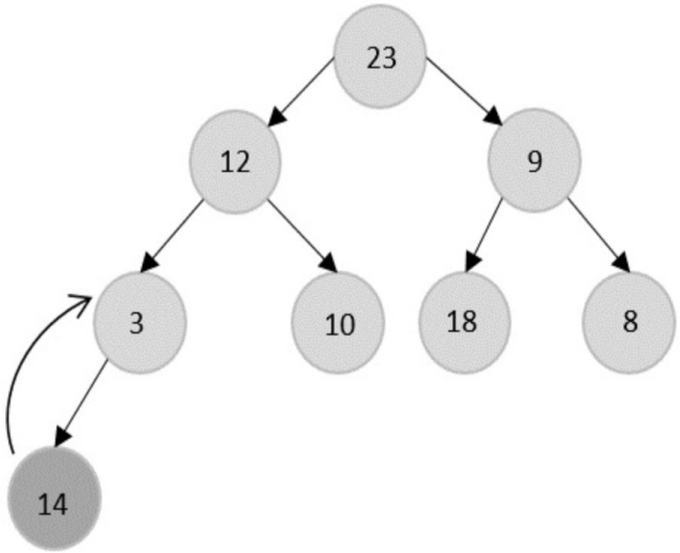
An example array to understand the sort algorithm − 12, 3, 9, 14, 10,18, 8, 23



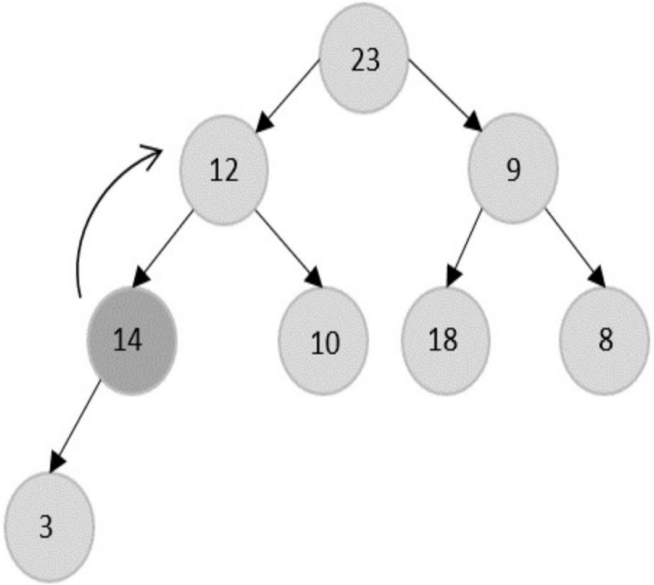


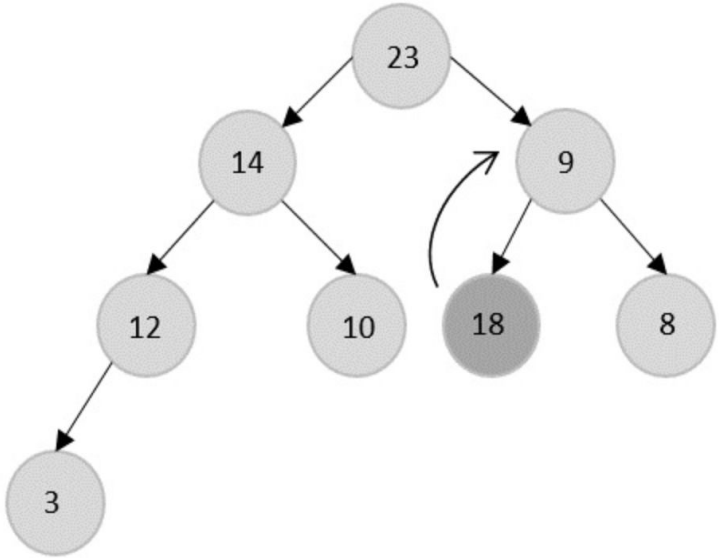
Heap Sort Example



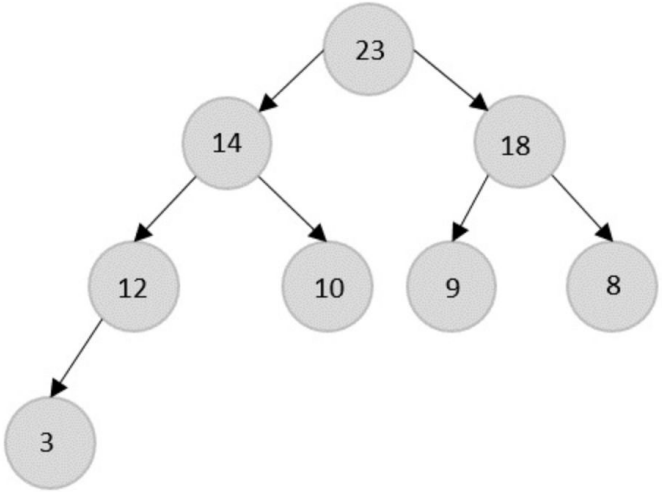


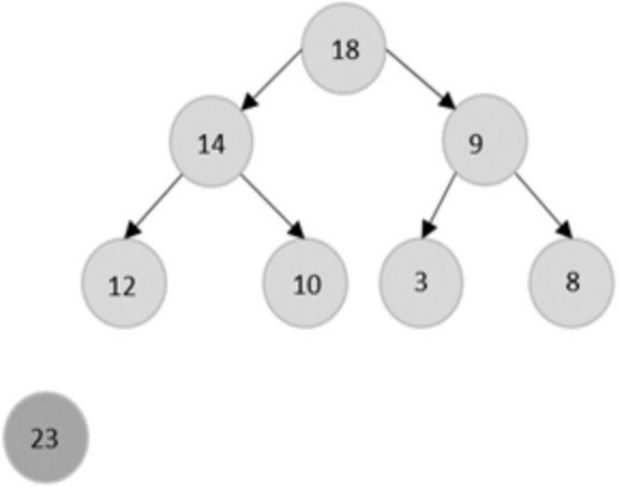
Heap Sort Example



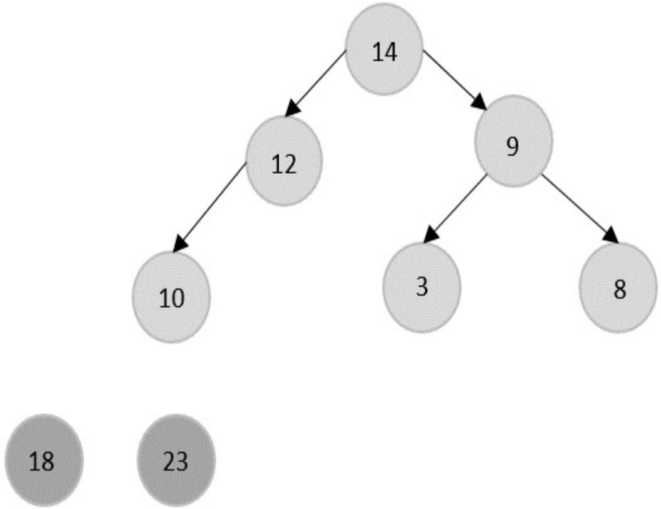


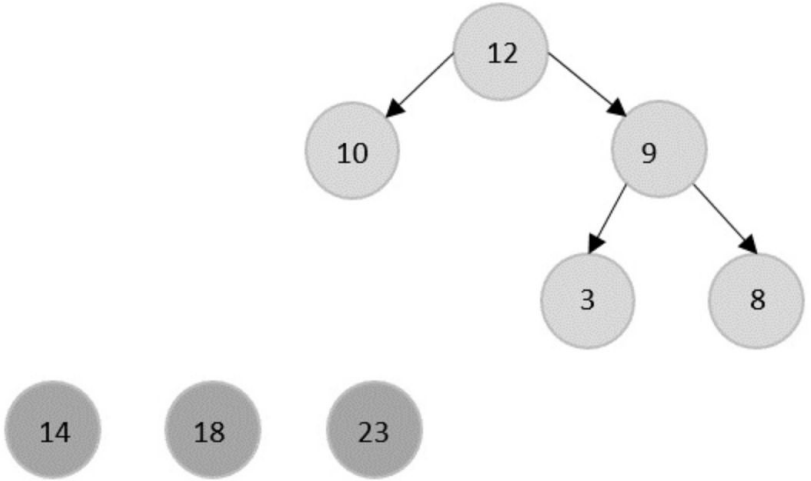
Heap Sort Example



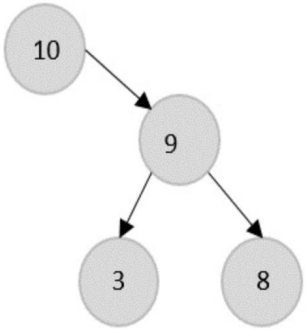


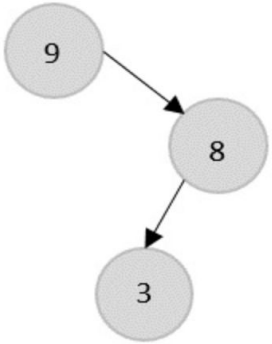
Heap Sort Example



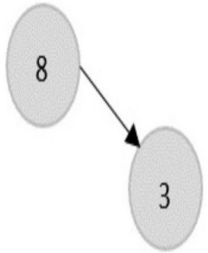


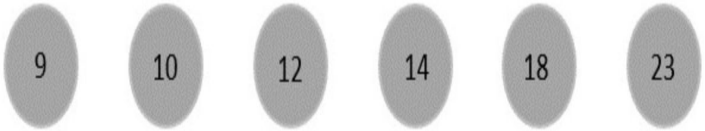
Heap Sort Example





Heap Sort Example





Heap Sort Example

Complexity Analysis of Heap Sort

Time Complexity: O(n log n)

Auxiliary Space: O(log n)

Advantages of Counting Sort

1.Heap Sort has a time complexity of O(n log n) in all cases.

2.Memory usage can be minimal (by writing an iterative heapify() instead of a recursive one).

3.It is simpler to understand than other equally efficient sorting algorithms.

Disadvantages of Counting Sort

1.Heap sort is costly as the constants are higher compared to merge sort even if the time complexity is O(n Log n) for both.

2.Heap sort is unstable. It might rearrange the relative order.

3.Heap Sort is not very efficient because of the high constants in the time complexity.

Thanks!!